7.3.4 AVL 树的删除

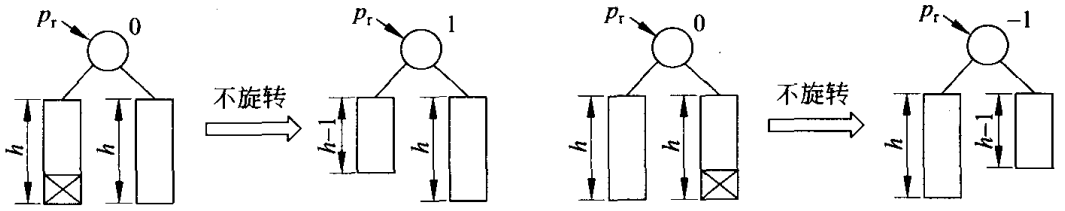
AVL 树的删除算法与二叉搜索树类似。不同之处在于：若删除后破坏了 AVL 树的高度平衡性质，还需要做平衡化旋转。

1. 如果被删结点 p 有两个子女。首先搜索 p 在中序次序下的直接前驱 q（同样可以找直接后继）。再把结点 q 的内容传送给结点 p，现在问题转移到删除结点 q。把结点 q 当作被删结点 p，它是只有一个子女的结点。

2. 如果被删结点 p 最多只有一个子女 q。可以简单地把 p 的父结点 p_r 中原来指向 p 的指针改指到 q；如果结点 p 没有子女，p 父结点 p_r 的相应指针置为 NULL。然后将原来以结点 p_r 为根的子树的高度减 1，并沿 p_r 通向根的路径反向追踪高度的这一变化对路径上各个结点的影响。

考查结点 q 的父结点 p_r。若 q 是 p_r 的左子女，则 p_r 的平衡因子应当增加 1，否则减少 1。根据修改后的 p_r 的平衡因子值，按三种情况分别进行处理：

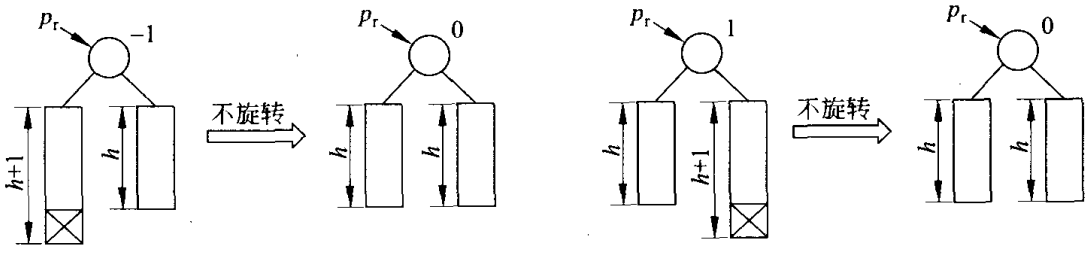
(1) p_r 的平衡因子原来为 0，在它的左子树或右子树被缩短后，则它的平衡因子改为 1 或 −1。由于以 p_r 为根的子树高度没有改变，从 p_r 到根结点的路径上所有结点都不需要调整。此时可结束本次删除的重新平衡过程。参看图 7.20。



(a) p_r的左子树上删除　　(b) p_r的右子树上删除

图 7.20　不旋转且无需平衡化旋转的情形

(2) 结点 p_r 的平衡因子原不为 0，且较高的子树被缩短，则 p 的平衡因子改为 0。此时以 p_r 为根的子树平衡，但其高度减 1。为此需要继续考查结点 p_r 的父结点的平衡状态。参看图 7.21。



(a) p_r的左子树上删除　　(b) p_r的右子树上删除

图 7.21　不平衡但需继续向上做平衡化处理

(3) 结点 p_r 的平衡因子原不为 0，且较矮的子树又被缩短，则在结点 p_r 发生不平衡。为此需进行平衡化旋转来恢复平衡。令 p_r 的较高的子树的根为 q（该子树未被缩短），根据 q 的平衡因子，有如下 3 种平衡化操作。

① 如果 q 的平衡因子为 0，执行一个单旋转来恢复结点 p_r 的平衡，如图 7.22 所示。图

中是左单旋转的例子，右单旋转的情形可以对称地处理。由于平衡旋转后以 q 为根的子树的高度没有发生改变，从 q 到根的路径上所有结点的平衡因子不变。此时可以结束重新平衡的过程。

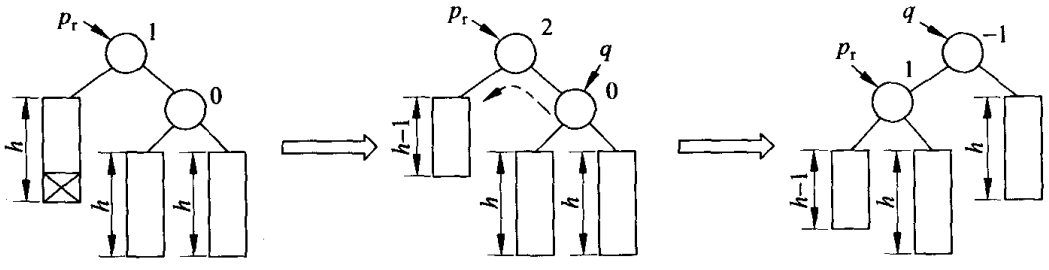


图 7.22　第一种单旋转平衡化处理

② 如果 q 的平衡因子与 p_r 的平衡因子（正负）号相同，则执行一个单旋转来恢复平衡，结点 p_r 和 q 的平衡因子均改为 0。如图 7.23 所示。图中是左单旋转的例子，右单旋转的情形可以对称地处理。由于经过平衡旋转后结点 q 的子树高度降低 1，故需要继续沿插入路径向上考查结点 q 的父结点的平衡状态，即将当前考查结点向根结点方向上移。

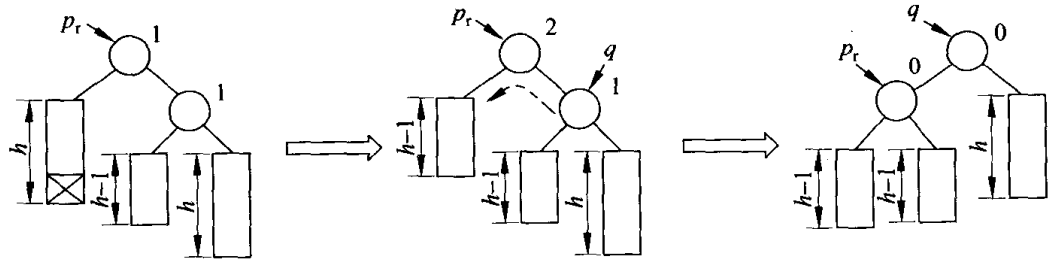


图 7.23　第二种单旋转平衡化处理

③ 如果 p_r 与 q 的平衡因子（正负）号相反，则执行一个双旋转来恢复平衡，先围绕 q 转再围绕 p_r 转。新的根结点的平衡因子置为 0，其他结点的平衡因子相应处理，同时由于经过平衡化处理后子树的高度降低 1，还需要考查它的父结点，继续向上层进行平衡化工作。参看图 7.24。

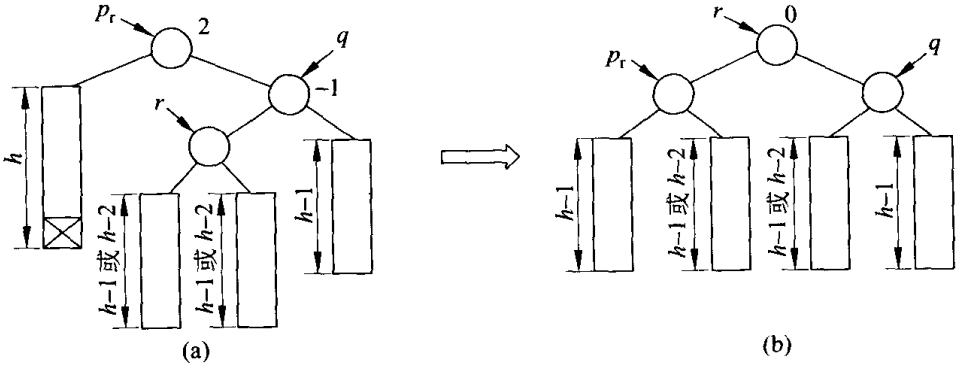


图 7.24　双旋转平衡化处理

程序 7.22 是删除算法，其中用到一个栈，在搜索被删除结点时记忆从根到被删除结点的路径，以便从下向上进行平衡化。